

29 Sep '26

PL/SQL notes

Revision for the Essex coding assessment (65 min, one attempt). Tables used in every example: `emp(emp_id, name, salary, dept_id, manager_id, hire_date)` and `dept(dept_id, dept_name)`.

Contents

1. Block structure
2. Variables & anchored types
3. SELECT INTO
4. IF, CASE & loops
5. Cursors
6. Exceptions
7. Procedures & functions
8. Packages
9. Triggers
10. Collections & bulk processing
11. Dynamic SQL
12. Transactions
13. SQL they will probably test
14. Classic coding programs
15. Rapid-fire facts
16. Exam-day checklist

1. Block structure

Every PL/SQL program is a block. Only **BEGIN ... END** is mandatory.

```
DECLARE                                -- optional: variables, cursors, types
  v_name emp.name%TYPE;
BEGIN                                  -- mandatory: the logic
  SELECT name INTO v_name
  FROM emp WHERE emp_id = 100;
  DBMS_OUTPUT.PUT_LINE('Hi ' || v_name);
EXCEPTION                             -- optional: error handling
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No such employee');
END;
/
```

- Run `SET SERVEROUTPUT ON` first or you see no output.
- Anonymous block = no name, not stored. Named blocks = procedure, function, package, trigger (stored in the DB).

- `:=` assigns, `=` compares, `||` joins strings.

2. Variables & anchored types

```
DECLARE
  v_sal    emp.salary%TYPE;           -- same type as the column
  r_emp    emp%ROWTYPE;               -- a whole row; use r_emp.name
  c_gst    CONSTANT NUMBER := 0.18;  -- must be initialised
  v_count  PLS_INTEGER NOT NULL := 0; -- NOT NULL needs a value too
  v_ok     BOOLEAN DEFAULT TRUE;

  TYPE t_emp IS RECORD (id NUMBER, name VARCHAR2(50));
  r_small  t_emp;
BEGIN
  SELECT * INTO r_emp FROM emp WHERE emp_id = 100;
  r_small.id := r_emp.emp_id;
END;
```

Trap! `%TYPE` means your code survives if the column changes from `NUMBER(6)` to `NUMBER(8)`. Interviewers love asking "why use `%TYPE`?"

- `BOOLEAN` can't go straight into `DBMS_OUTPUT.PUT_LINE`; convert with a `CASE`.
- `VARCHAR2` in PL/SQL can hold up to 32767 bytes; a table column is 4000 by default.

3. SELECT INTO

Must return **exactly one row**.

- 0 rows → `NO_DATA_FOUND`
- 2+ rows → `TOO_MANY_ROWS`

Trap! `SELECT COUNT(*) INTO v FROM emp WHERE 1=0;` does NOT raise `NO_DATA_FOUND`. Aggregates always return one row (here 0).

4. IF, CASE & loops

```
IF v_sal > 50000 THEN
  v_band := 'HIGH';
ELSIF v_sal > 20000 THEN      -- ELSIF, not ELSEIF / ELSE IF
  v_band := 'MID';
ELSE
  v_band := 'LOW';
END IF;

v_band := CASE                -- searched CASE expression
  WHEN v_sal > 50000 THEN 'HIGH'
  WHEN v_sal > 20000 THEN 'MID'
  ELSE 'LOW'
END;
```

Trap! A CASE statement (ends with END CASE;) with no matching WHEN and no ELSE raises CASE_NOT_FOUND. A CASE expression just returns NULL.

```
-- 1. basic loop: runs at least once
LOOP
  i := i + 1;
  EXIT WHEN i >= 5;
END LOOP;

-- 2. while loop: checks before each pass
WHILE i <= 5 LOOP
  i := i + 1;
END LOOP;

-- 3. numeric for loop: i is auto-declared and read-only
FOR i IN 1..5 LOOP
  CONTINUE WHEN MOD(i, 2) = 0;    -- skip evens
  DBMS_OUTPUT.PUT_LINE(i);       -- 1 3 5
END LOOP;

FOR i IN REVERSE 1..5 LOOP ...   -- 5,4,3,2,1 (still write low..high!)
```

Implicit cursor (every DML / SELECT INTO)

Attribute	Meaning
SQL%ROWCOUNT	rows touched by the last statement
SQL%FOUND / SQL%NOTFOUND	did it touch any row?
SQL%ISOPEN	always FALSE for implicit cursors

Explicit cursor: OPEN, FETCH, CLOSE

```
DECLARE
    CURSOR c_emp IS
        SELECT name, salary FROM emp WHERE dept_id = 50;
    v_name emp.name%TYPE;
    v_sal emp.salary%TYPE;
BEGIN
    OPEN c_emp;
    LOOP
        FETCH c_emp INTO v_name, v_sal;
        EXIT WHEN c_emp%NOTFOUND;           -- immediately after FETCH
        DBMS_OUTPUT.PUT_LINE(v_name || ' ' || v_sal);
    END LOOP;
    CLOSE c_emp;
END;
```

Cursor FOR loop (the one to use in exams)

```
BEGIN
  FOR r IN (SELECT name, salary FROM emp WHERE dept_id = 50) LOOP
    DBMS_OUTPUT.PUT_LINE(r.name || ' earns ' || r.salary);
  END LOOP;
END;
```

Parameterised cursor + FOR UPDATE

```
DECLARE
  CURSOR c_dept (p_dept NUMBER) IS
    SELECT emp_id, salary FROM emp
    WHERE dept_id = p_dept
    FOR UPDATE OF salary;           -- locks the rows
BEGIN
  FOR r IN c_dept(50) LOOP
    UPDATE emp SET salary = r.salary * 1.05
    WHERE CURRENT OF c_dept;       -- updates the row just fetched
  END LOOP;
  COMMIT;
END;
```

- `SYS_REFCURSOR` = a cursor variable; use it to return a result set from a procedure (`OPEN p_rc FOR SELECT ...`).
- Opening an already open cursor → `CURSOR_ALREADY_OPEN`. Fetching a closed one → `INVALID_CURSOR`.

6. Exceptions

Predefined	Error	When
<code>NO_DATA_FOUND</code>	<code>ORA-01403</code>	<code>SELECT INTO</code> got 0 rows
<code>TOO_MANY_ROWS</code>	<code>ORA-01422</code>	<code>SELECT INTO</code> got 2+ rows
<code>ZERO_DIVIDE</code>	<code>ORA-01476</code>	divide by zero
<code>VALUE_ERROR</code>	<code>ORA-06502</code>	value too big / bad conversion in PL/SQL
<code>INVALID_NUMBER</code>	<code>ORA-01722</code>	bad string-to-number inside SQL
<code>DUP_VAL_ON_INDEX</code>	<code>ORA-00001</code>	unique / primary key violated

```

DECLARE
    e_low_sal EXCEPTION;           -- user-defined
    e_child_rows EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_child_rows, -2292); -- name an ORA error
    v_sal emp.salary%TYPE;
BEGIN
    SELECT salary INTO v_sal FROM emp WHERE emp_id = 100;
    IF v_sal < 1000 THEN
        RAISE e_low_sal;
    END IF;
EXCEPTION
    WHEN e_low_sal THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary below minimum');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee missing');
    WHEN OTHERS THEN               -- always last
        DBMS_OUTPUT.PUT_LINE(SQLCODE || ':' || SQLERRM);
        RAISE;                     -- re-raise, don't swallow
END;

```

- `RAISE_APPLICATION_ERROR` codes must be between `-20000` and `-20999`.
- `SQLCODE` is `+100` for `NO_DATA_FOUND` and `+1` for user-defined exceptions.
- Unhandled errors propagate to the enclosing block, then to the caller.

Trap! An error raised in the `DECLARE` section (e.g. `v.NUMBER(2).:= 999;`) is NOT caught by that block's own `EXCEPTION` section. It goes straight to the outer block.

7. Procedures & functions

```
CREATE OR REPLACE PROCEDURE raise_salary (  
  p_emp_id IN emp.emp_id%TYPE,  
  p_pct    IN  NUMBER DEFAULT 10,  
  p_new    OUT emp.salary%TYPE  
) AS  
BEGIN  
  UPDATE emp  
    SET salary = salary * (1 + p_pct / 100)  
    WHERE emp_id = p_emp_id  
    RETURNING salary INTO p_new;  
  
  IF SQL%ROWCOUNT = 0 THEN  
    RAISE_APPLICATION_ERROR(-20002, 'Employee ' || p_emp_id || ' not found');  
  END IF;  
END raise_salary;  
/  
  
-- calling it  
DECLARE  
  v_new emp.salary%TYPE;  
BEGIN  
  raise_salary(p_emp_id => 100, p_new => v_new);  -- named notation  
  DBMS_OUTPUT.PUT_LINE('New salary: ' || v_new);  
END;
```

```
CREATE OR REPLACE FUNCTION annual_sal (p_id IN emp.emp_id%TYPE)  
RETURN NUMBER  
IS  
  v_sal emp.salary%TYPE;  
BEGIN  
  SELECT salary INTO v_sal FROM emp WHERE emp_id = p_id;  
  RETURN v_sal * 12;  
EXCEPTION  
  WHEN NO_DATA_FOUND THEN RETURN NULL;  
END;  
/  
  
SELECT name, annual_sal(emp_id) FROM emp;  -- functions work inside SQL
```

	<i>Procedure</i>	<i>Function</i>
Returns	nothing directly; uses OUT params	must RETURN exactly one value
Called as	a statement	part of an expression
Usable in SELECT?	no	yes, if it does no DML and has no OUT params
Typical use	do work (DML, business logic)	compute and return a value

- Modes: *IN* (default, read-only), *OUT* (starts as NULL, write back), *IN OUT* (both).
- *NOCOPY* hint passes big OUT params by reference for speed.

8. Packages

Spec = the public "menu". Body = the implementation. Change the body without recompiling callers.

```

CREATE OR REPLACE PACKAGE emp_pkg AS
    g_bonus_pct NUMBER := 5;                -- public, lives for the session
    FUNCTION headcount (p_dept NUMBER) RETURN NUMBER;
    PROCEDURE give_bonus (p_dept NUMBER);
END emp_pkg;
/

CREATE OR REPLACE PACKAGE BODY emp_pkg AS
    FUNCTION headcount (p_dept NUMBER) RETURN NUMBER IS
        v NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v FROM emp WHERE dept_id = p_dept;
        RETURN v;
    END;

    PROCEDURE give_bonus (p_dept NUMBER) IS
    BEGIN
        UPDATE emp SET salary = salary * (1 + g_bonus_pct / 100)
        WHERE dept_id = p_dept;
    END;
END emp_pkg;
/

EXEC emp_pkg.give_bonus(50);

```

- Why packages: grouping, hiding private code, **overloading**, session-level global variables, loaded into memory once.
- Anything only in the body is private.

9. Triggers

```
CREATE OR REPLACE TRIGGER trg_sal_audit
AFTER UPDATE OF salary ON emp
FOR EACH ROW                                -- row-level
WHEN (NEW.salary <> OLD.salary)              -- no colon inside WHEN
BEGIN
    INSERT INTO sal_audit (emp_id, old_sal, new_sal, changed_on)
    VALUES (:OLD.emp_id, :OLD.salary, :NEW.salary, SYSDATE);
END;
/

CREATE OR REPLACE TRIGGER trg_emp_check
BEFORE INSERT OR UPDATE ON emp
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        :NEW.hire_date := NVL(:NEW.hire_date, SYSDATE);
    END IF;
    IF :NEW.salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20010, 'Salary cannot be negative');
    END IF;
END;
/
```

- **:OLD** is NULL on INSERT, **:NEW** is NULL on DELETE.
- **BEFORE** triggers can change **:NEW** values; **AFTER** triggers can't.
- Statement-level (no **FOR EACH ROW**) fires once per statement, even for 0 rows.
- **INSTEAD OF** triggers are for views.
- No **COMMIT** inside a trigger (unless it's an autonomous transaction).

Trap! A row-level trigger that queries its own table gives ORA-04091: mutating table. Fix: compound trigger, or move logic to a statement-level trigger.

10. Collections & bulk processing

Type	Declare	Notes
Associative array	TABLE OF NUMBER INDEX BY PLS_INTEGER	no init needed, sparse, PL/SQL only, can index by VARCHAR2
Nested table	TABLE OF NUMBER	needs constructor t() + EXTEND, unbounded, storable in a column
VARRAY	VARRAY(10) OF NUMBER	fixed max size, always dense, keeps order

Methods: COUNT FIRST LAST NEXT PRIOR EXISTS EXTEND TRIM DELETE LIMIT

```
DECLARE
  TYPE t_ids IS TABLE OF emp.emp_id%TYPE;
  l_ids t_ids;
BEGIN
  SELECT emp_id BULK COLLECT INTO l_ids      -- one trip to SQL engine
  FROM emp WHERE dept_id = 10;

  FORALL i IN 1 .. l_ids.COUNT              -- one trip back
    UPDATE emp SET salary = salary * l.i
    WHERE emp_id = l_ids(i);

  DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows updated');
END;
```

Why bulk: fewer **context switches** between the PL/SQL and SQL engines, much faster than row-by-row.

```
-- big tables: fetch in batches with LIMIT
OPEN c;
LOOP
  FETCH c BULK COLLECT INTO l_rows LIMIT 1000;
  EXIT WHEN l_rows.COUNT = 0;      -- NOT c%NOTFOUND here
  FORALL i IN 1 .. l_rows.COUNT
    INSERT INTO emp_archive VALUES l_rows(i);
END LOOP;
CLOSE c;
```

Trap! `EXIT WHEN c%NOTFOUND` right after a `LIMIT` fetch skips the last partial batch. Check `,COUNT = 0` instead. Also: `BULK COLLECT` never raises `NO_DATA_FOUND`.

`FORALL ... SAVE EXCEPTIONS` keeps going on bad rows; read them afterwards from `SQL%BULK_EXCEPTIONS` (error `ORA-24381`).

11. Dynamic SQL

```
-- table name only known at run time
EXECUTE IMMEDIATE 'SELECT COUNT(*) FROM ' || DBMS_ASSERT.SQL_OBJECT_NAME(p_table)
  INTO v_cnt;

-- bind variables for values: fast and safe from SQL injection
EXECUTE IMMEDIATE 'UPDATE emp SET salary = :1 WHERE emp_id = :2'
  USING v_sal, v_id;

-- DDL inside PL/SQL must be dynamic
EXECUTE IMMEDIATE 'CREATE TABLE tmp_ids (id NUMBER)';
```

- You can bind values, but never table or column names; validate those (`DBMS_ASSERT`).

12. Transactions

```
INSERT INTO dept VALUES (99, 'Analytics');
SAVEPOINT after_dept;
UPDATE emp SET dept_id = 99 WHERE emp_id = 100;
ROLLBACK TO after_dept;    -- undoes only the UPDATE
COMMIT;                   -- dept 99 is saved
```

- Any DDL (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`) does an **implicit COMMIT**.
- `PRAGMA AUTONOMOUS_TRANSACTION`: an independent transaction, e.g. an error-log procedure that commits even if the caller rolls back. It must `COMMIT` or `ROLLBACK` itself.

13. SQL they will probably test

Analyst assessments lean heavily on plain SQL. Know these cold.

Logical order a query runs in

FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → FETCH

So a SELECT alias can't be used in WHERE, but can be used in ORDER BY.

Nth highest salary

```
SELECT DISTINCT salary
FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM emp
)
WHERE rnk = 3;
```

Top earner in each department

```
SELECT dept_id, name, salary
FROM (
  SELECT e.*,
         ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
  FROM emp e
)
WHERE rn = 1;      -- use RANK() if ties should all show
```

salary	ROW_NUMBER	RANK	DENSE_RANK
900	1	1	1
900	2	1	1
800	3	3	2

Find and delete duplicates

```
SELECT name, dept_id, COUNT(*)
FROM emp
GROUP BY name, dept_id
HAVING COUNT(*) > 1;

DELETE FROM emp
WHERE ROWID NOT IN (
    SELECT MIN(ROWID) FROM emp GROUP BY name, dept_id
);
-- keeps one copy of each
```

Running total and change from previous row

```
SELECT hire_date, salary,
       SUM(salary) OVER (ORDER BY hire_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_total,
       salary - LAG(salary) OVER (ORDER BY hire_date) AS diff_prev
FROM emp;
```

Employees earning more than their manager (self join)

```
SELECT e.name, e.salary, m.name AS manager, m.salary AS mgr_salary
FROM emp e
JOIN emp m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Departments with no employees

```
SELECT d.dept_name
FROM dept d
LEFT JOIN emp e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

Paid above their department's average (correlated subquery)

```
SELECT name, salary, dept_id
FROM emp e
WHERE salary > (SELECT AVG(salary) FROM emp WHERE dept_id = e.dept_id);
```

Upsert with MERGE

```
MERGE INTO emp t
USING emp_stage s
ON (t.emp_id = s.emp_id)
WHEN MATCHED THEN
  UPDATE SET t.salary = s.salary
WHEN NOT MATCHED THEN
  INSERT (emp_id, name, salary, dept_id)
  VALUES (s.emp_id, s.name, s.salary, s.dept_id);
```

Top-N and the ROWNUM trap

```
-- WRONG: ROWNUM is assigned before ORDER BY
SELECT * FROM emp WHERE ROWNUM <= 3 ORDER BY salary DESC;

-- RIGHT
SELECT * FROM (SELECT * FROM emp ORDER BY salary DESC) WHERE ROWNUM <= 3;

-- RIGHT (12c+)
SELECT * FROM emp ORDER BY salary DESC FETCH FIRST 3 ROWS ONLY;
```

Trap! `WHERE ROWNUM = 2` returns nothing: the first row gets ROWNUM 1, fails, and so ROWNUM never reaches 2.

NULL traps

- `NULL = NULL` is not TRUE. Use `IS NULL` / `IS NOT NULL`.
- `NOT IN (subquery)` returns no rows if the subquery has a NULL. Use `NOT EXISTS`.
- `COUNT(*)` counts all rows; `COUNT(col)` skips NULLs; `AVG/SUM` ignore NULLs.
- In Oracle an empty string " IS NULL.
- `NVL(a,b)`, `NVL2(a, if_not_null, if_null)`, `COALESCE(a,b,c)` (first non-null), `NULLIF(a,b)` (NULL if equal, handy to avoid divide-by-zero).

Function cheat sheet

Kind	Functions
Dates	<code>SYSDATE</code> , <code>TRUNC(d,'MM')</code> , <code>ADD_MONTHS</code> , <code>MONTHS_BETWEEN</code> , <code>LAST_DAY</code> , <code>EXTRACT(YEAR FROM d)</code> ; <code>date - date = days</code>
Strings	<code>SUBSTR</code> , <code>INSTR</code> , <code>LENGTH</code> , <code>UPPER/LOWER/INITCAP</code> , <code>TRIM</code> , <code>LPAD/RPAD</code> , <code>REPLACE</code> , <code>REGEXP_LIKE</code> , <code>REGEXP_REPLACE</code>
Convert	<code>TO_CHAR(d,'YYYY-MM')</code> , <code>TO_DATE('2026-09-29','YYYY-MM-DD')</code> , <code>TO_NUMBER</code> , <code>CAST</code>
Aggregate	<code>LISTAGG(name, ' ') WITHIN GROUP (ORDER BY name)</code> , <code>COUNT</code> , <code>SUM</code> , <code>AVG</code> , <code>MIN</code> , <code>MAX</code>
Branch	<code>CASE WHEN ... END</code> , <code>DECODE(col, 1,'A', 2,'B', 'other')</code>

DELETE vs TRUNCATE vs DROP

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
WHERE allowed	yes	no	no
Rollback	yes	no (auto-commit)	no (recycle bin: FLASHBACK)
Fires DML triggers	yes	no	no
Table structure	kept	kept	gone

14. Classic coding programs

Reverse a string / palindrome

```
DECLARE
  s VARCHAR2(50) := 'MADAM';
  r VARCHAR2(50);
BEGIN
  FOR i IN REVERSE 1 .. LENGTH(s) LOOP
    r := r || SUBSTR(s, i, 1);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Reversed: ' || r);
  IF UPPER(s) = UPPER(r) THEN
    DBMS_OUTPUT.PUT_LINE(s || ' is a palindrome');
  ELSE
    DBMS_OUTPUT.PUT_LINE(s || ' is not a palindrome');
  END IF;
END;
```

Factorial

```
DECLARE
  n NUMBER := 5;
  f NUMBER := 1;
BEGIN
  FOR i IN 1 .. n LOOP
    f := f * i;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(n || '! = ' || f);    -- 5! = 120
END;
```

Fibonacci (first 10)

```
DECLARE
  a NUMBER := 0;
  b NUMBER := 1;
  t NUMBER;
BEGIN
  FOR i IN 1 .. 10 LOOP
    DBMS_OUTPUT.PUT(a || ' ');
    t := a + b;
    a := b;
    b := t;
  END LOOP;
  DBMS_OUTPUT.NEW_LINE;
END;
```

Prime check

```
DECLARE  
    n          NUMBER := 29;  
    is_prime   BOOLEAN := n > 1;  
  
BEGIN  
    FOR i IN 2 .. TRUNC(SQRT(n)) LOOP  
        IF MOD(n, i) = 0 THEN  
            is_prime := FALSE;  
            EXIT;  
        END IF;  
    END LOOP;  
  
    DBMS_OUTPUT.PUT_LINE(n || CASE WHEN is_prime THEN ' is prime'  
                                   ELSE ' is not prime' END);  
  
END;
```

Sum of digits

```
DECLARE
  n NUMBER := 4567;
  s NUMBER := 0;
BEGIN
  WHILE n > 0 LOOP
    s := s + MOD(n, 10);
    n := TRUNC(n / 10);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Sum = ' || s);      -- Sum = 22
END;
```

Department report with a cursor (common "real" task)

```
BEGIN
  FOR d IN (SELECT dept_id, dept_name FROM dept ORDER BY dept_id) LOOP
    DBMS_OUTPUT.PUT_LINE('--- ' || d.dept_name || ' ---');
    FOR e IN (SELECT name, salary FROM emp
              WHERE dept_id = d.dept_id ORDER BY salary DESC) LOOP
      DBMS_OUTPUT.PUT_LINE(RPAD(e.name, 20) || e.salary);
    END LOOP;
  END LOOP;
END;
```

15. Rapid-fire facts

- `%TYPE` = one column's type; `%ROWTYPE` = a whole row.
- Implicit cursor = Oracle opens it for every DML; explicit = you declare and control it.
- `WHERE` filters rows before grouping; `HAVING` filters groups after.
- `UNION` removes duplicates (sorts, slower); `UNION ALL` keeps them. Oracle uses `MINUS` (others: `EXCEPT`).
- Overloading = same name, different parameters; only in packages (or nested subprograms).
- Package variables keep their value for the whole session.
- `PLS_INTEGER` is faster than `NUMBER` for integer counters.
- `DBMS_OUTPUT` text appears only after the block finishes.
- A function called from `SELECT` can't do DML (ORA-14551).
- Views don't store data; materialized views do.

- Index helps WHERE/JOIN lookups; wrapping the column in a function (`UPPER(name)`) stops a normal index being used.
- If the test isn't Oracle: `NVL` → `COALESCE`, `MINUS` → `EXCEPT`, `ROWNUM` → `LIMIT`, `||` may be `CONCAT`.

16. Exam-day checklist

- Link expires 1 Oct, around 6:06 PM IST. Don't leave it to the last hour.
- 65 minutes, one attempt, one sitting. The timer doesn't pause if you disconnect.
- Chrome or Edge on a laptop. Unplug any second monitor. Webcam on, face lit, head and shoulders in frame.
- Be alone, stay in the tab, and don't paste code in: tab switches and copy/paste are logged.
- Skim every question first, bank the easy ones, then come back.
- Before submitting SQL, check ties, NULLs, empty results, and whether they asked for `DISTINCT`.
- Write readable code: aliases, indentation, a one-line comment on tricky logic.

You've got this, Eshan!